

P2455R0

2021 November Library Evolution Poll Outcomes

Published Proposal, 2021-12-08

Author:

[Bryce Adelstein Leibach \(he/him/his\)](#) — [Library Evolution Chair](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

- 3.1 Poll 1: [P2465R1] Standard Library Modules `std` And `std.compat`
- 3.2 Poll 2: [P2387R2] Pipe Support For User-Defined Range Adaptors
- 3.3 Poll 3: [P2443R0] `views::chunk_by`
- 3.4 Poll 4: [P2442R0] Windowing Range Adaptors: `views::chunk` And `views::slide`
- 3.5 Poll 5: [P2440R0] `ranges::iota`, `ranges::shift_left`, And `ranges::shift_right`
- 3.6 Poll 6: [P2255R2] A Type Trait To Detect Reference Binding To Temporary
- 3.7 Poll 7: [P1885R8] Naming Text Encodings To Demystify Them

- 3.8 Poll 8: [P2419R1] Clarify Handling Of Encodings In Localized Formatting Of Chrono Types
- 3.9 Poll 9: [P2460R0] Relax Requirements On `wchar_t` To Match Existing Practices
- 3.10 Poll 10: [P2445R0] `forward_like`
- 3.11 Poll 11: [P2417R0] A More `constexpr` `bitset`
- 3.12 Poll 12: [P1841R1] Wording For Individually Specializable Numeric Traits
- 3.13 Poll 13: [P0627R6] Function To Mark Unreachable Code

References

Informative References

§ 1. Introduction

In November 2021, the C++ Library Evolution group conducted a series of electronic decision polls [\[P2454R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 37 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send [P2465R1] (Standard Library Modules <code>std</code> And <code>std.compat</code>) to Library Working Group for C++23, classified as a focus ([P0592R4] bucket 1 item).	20	9	1	2	1	Consensus in favor.
Poll 2: Send [P2387R2] (Pipe Support For User-Defined Range Adaptors) to Library Working	15	9	0	0	0	Unanimous consensus in favor.

Group for C++23, classified as an addition ([P0592R4] bucket 3 item).						
Poll 3: Send [P2443R0] (<code>views::chunk_by</code>) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	14	5	0	0	0	Unanimous consensus in favor.
Poll 4: Modify [P2442R0] (Windowing Range Adaptors: <code>views::chunk</code> And <code>views::slide</code>) by replacing the current feature test macro with separate feature test macros for <code>views::chunk</code> and <code>views::slide</code> and then send the revised paper to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	14	4	0	0	0	Unanimous consensus in favor.
Poll 5: Send [P2440R0] (<code>ranges::iota</code> , <code>ranges::shift_left</code> , And <code>ranges::shift_right</code>) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	11	12	0	0	0	Unanimous consensus in favor.
Poll 6: Send [P2255R2] (A Type Trait To Detect Reference Binding To Temporary) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	14	12	1	0	0	Strong consensus in favor.
Poll 7: Send [P1885R8] (Naming Text Encodings To Demystify Them) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	11	10	2	2	2	Consensus in favor.
Poll 8: Send [P2419R1] (Clarify Handling Of Encodings In Localized Formatting Of Chrono Types) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	13	11	1	0	0	Strong consensus in favor.
Poll 9: Send [P2460R0] (Relax Requirements On <code>wchar_t</code> To Match Existing Practices) to Library Working Group for C++23, classified as	12	15	1	1	0	Consensus in favor.

an improvement of an existing feature ([P0592R4] bucket 2 item).						
Poll 10: Send [P2445R0] (forward_like) to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	14	11	1	0	0	Strong consensus in favor.
Poll 11: Send [P2417R0] (A More constexpr bitset) to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).	13	13	3	0	0	Strong consensus in favor.
Poll 12: Modify [P1841R1] (Wording For Individually Specializable Numeric Traits) by applying [P2485R0] (Do Not Add value_exists And value_or To C++23), and then send the revised paper to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	13	8	0	0	0	Unanimous consensus in favor.
Poll 13: Send [P0627R6] (Function To Mark Unreachable Code), which was previously approved and has been modified to remove the message parameter, to Library Working Group for C++23, classified as an addition ([P0592R4] bucket 3 item).	27	4	1	0	0	Strong consensus in favor.

§ 3. Selected Poll Comments

§ 3.1. Poll 1: [P2465R1] Standard Library Modules std And std.compat

Given the performance result, I believe this paper provides right solution. Also, I believe having both std and std.compat modules, is necessary for preserving namespace hygiene. The alternative of `import std;` followed by `using namespace std;` is strictly worse.

— Strongly Favor

Simplicity argues in favor of providing this feature even if we later provide smaller modules.

While the idea of having `std.foo` be such a subset of `std` is a good one in general (partly because it might serve to encourage similar arrangements in user-defined libraries), there is no current plan to create them, and if any come to be it will not be fatal to have `std.compat` be one exception to the rule.

— Strongly Favor

We need something like this badly. It makes sense that there are two modules. I agree that the "cleaner" option should be easier to type. I also agree that the "make everything work" option should be a single line of code, and not two lines (as in a separate module for the global namespace).

There was a debate over whether the period (as in `std.compat`) should imply an increase in specificity or hierarchy depth. In this case, `std.compat` does more importing than `std`, so it is less specific. This might surprise some users, whose experience in other programming languages with modules might lead them to expect periods to imply increasing hierarchy depth or specificity. However, I think having the `import std` feature is worth this minor risk of confusion. C++ does not tie modules to the filesystem or a namespace hierarchy, so there's no reason to make periods significant.

The `import std` statement being fewer characters will encourage users to type `std::` in front of functions and types. This will make C++ more robust to any future incompatible changes in the C Standard. It will also make teaching C++ easier, by removing special cases (if it's in the Standard Library, it lives in the `std` namespace). I would hope that coding standards discourage `import std.compat` in favor of `import std`."

— Strongly Favor

I'm not happy with the name `std.compat`, but I can live with it. `stdcompat` would avoid the whole dot-confusion and I don't understand why people insist on the dot.

— Weakly Favor

I like the proposal for its simplicity and that we managed to for once give the good default syntax space to the modern best practice. I would have wished for a somewhat more in depth discussion of finer grained modules (along the lines of prerequisites/dependencies, i.e. `std.io`, `std.compiler_support`, `std.core`, `std.freestanding`, ...) but I am aware that we did not have time in the C++23 timeframe.

— Weakly Favor

It will finally provide the standard library module support that was missing from C++20. Voting weakly in favor because the current scheme makes it unnecessarily awkward to use types like `int32_t`.

— Weakly Favor

This is a kind of "bare-minimum" modularisation. It's hard to judge the value of this, but I suppose we need to do *something*.

— Neutral

By examples, this proposal destroys the possibility of having a reasonable meaning for dots in modules names.

— Did Not Participate

I am still unconvinced that it's a good idea to claim the unqualified `std` module name.

— Weakly Against

Grabbing `std` before we have any experience or plan for how module structure affects evolution of the library is reckless. We will regret this.

The reason to grab `std` now is frivolous.

"Can teach" `import std;`

vs

"Can't teach" `import std.all;`

— Strongly Against

§ 3.2. Poll 2: [P2387R2] Pipe Support For User-Defined Range Adaptors

This is an enabler. This takes of pressure of the standard as users (or third party providers) can now create their own ranges/views e.g. or also polyfills/backports of planned features. That this feature was missing from C++20 was one of my biggest fear when we standardized ranges. I am happy that we get it now.

— Strongly Favor

I believe this is most important range related paper, as it opens path for the libraries that defines views that can interoperate with standard ones. It allows us to escape "the standard should provide every view, otherwise they will not pipeable" trap.

— Strongly Favor

This is the piece that enables interop with standard range adaptor closure objects; the wording needs a bit more work, but the design is sound.

— Strongly Favor

The author carefully surveys different implementation approaches, and makes a reasonable decision. The only reason I'm not voting SF is because I'm not a ranges implementation expert.

The proposed `bind_back` function is a helpful utility that deserves standardization, just like `bind_front`. I appreciate how the author slotted the `bind_back` wording into the `bind_front` clause with minimal wording changes, and renamed the clause to describe both.

— Weakly Favor

The pipe syntax can't be taken seriously so long as it is restricted to standard-library adaptors; one can't write a template that applies an arbitrary adaptor using the syntax if it's not reliably available. However, I am not well-informed about the details of the design tradeoffs presented in the paper.

— Did Not Participate

§ 3.3. Poll 3: [P2443R0] `views::chunk_by`

This is an immediately useful feature that comes up regularly in intermediate students code.

— Strongly Favor

This is a quite useful range adaptor and has been proven in the field in ranges v3.

— Strongly Favor

This is a feature that will help make ranges more usable and complete in C++23.

— Strongly Favor

This is one of the many steps in filling out ranges.

— Weakly Favor

§ 3.4. Poll 4: [P2442R0] Windowing Range Adaptors: `views::chunk` And `views::slide`

A sensible and uncontroversial standard view.

— Strongly Favor

Useful functionality, well specified.

— Strongly Favor

Extending ranges will help more developers to be pulled towards it

— Strongly Favor

I have not followed this paper, although the semantics seem eminently obvious.

— Did Not Participate

§ 3.5. Poll 5: [P2440R0] `ranges::iota`, `ranges::shift_left`, And `ranges::shift_right`

There should be a range algorithm for every one of the iterator algorithms in the Standard Library. This is a step in that direction.

— Strongly Favor

These are features that will help make ranges more usable and complete in C++23. I'm convinced by the author's argument that `shift_left` and `shift_right` do not need to be changed to return the original end of the range.

— Strongly Favor

It is unfortunate to have duplicate versions of all algorithms, but it is a simplification for it to actually be all of them rather than most of them. The analysis of the `shift_left` return value is correct, even if the result is not perfectly easy to use in all cases.

— Weakly Favor

Good to continue rangifying the algorithms we missed.

— Weakly Favor

§ 3.6. Poll 6: [P2255R2] A Type Trait To Detect Reference Binding To Temporary

We have here a rare opportunity to actually do damage to the perennial dangling-reference pitfall in C++ without any false positives. Plausible language solutions would be either incomplete or no more automatic than using a trait.

— Strongly Favor

Lets us take code that currently dangles and make it not compile. Seems great. Would be better if this were unnecessary and instead somehow detectable by the language, but until that happens, this is a worthwhile substitute.

— Strongly Favor

This is a trait that is next to impossible to get right outside of the standard library, so it should definitely be provided by us.

— Weakly Favor

This fixes the reported problem. I'm not thrilled that we have to solve it this way, but I have no better suggestion either.

— Weakly Favor

This proposes a very clever facility that experts can use to protect against a small subset of value lifetime errors in C++ programs. It is one of a number of library and language papers which attempt to tackle C++'s lack of lifetime annotation or analysis in a piecemeal fashion. The new facility will help users avoid accidentally-dangling references, but at the cost of adding yet another complex subtlety for library authors.

— Neutral

§ 3.7. Poll 7: [P1885R8] Naming Text Encodings To Demystify Them

I strongly support this paper.

I implemented [P1885R8]'s backing compiler technology in all public-facing compilers I could get my hands on (Clang and its derivatives as well as GCC). I also - after much gnashing of teeth and over a year of waiting - got Microsoft to implement it and ship it in their first version of the VS2022 Visual C++ compiler. P1885 is an accurate capturing of the status quo for how encodings are detected on platforms. It is not the end-all be-all way to determine encoding names, but asking for anything more is - quite literally - impossible to force implementation's hands to do. This is rooted in the POSIX specification, the C specification, and several of the vendors who do not provide equivalent functionality to unambiguously ID the encodings used on those platforms and therefore must rely on a combination of names, platform hints, and more.

[P1885R8] takes care of providing portable names. For the **vast majority** of use cases, [P1885R8] gives the user a way to detect the encodings they could possibly care about, and allows them to act on that information (including at compile-time) in a way that makes it easy to write user-friendly libraries. I would know, because I had to spend an inordinate amount of time reimplementing exactly what was in [P1885R8] for both the compiler-side (literal encodings) and runtime (in my own now-deployed libraries). Platform hints still must be the domain of the user to take care of, wherein the subtleties of using e.g. and IBM-based CN or JP encoding versus a Microsoft-based CN or JP must be captured through platform-specific information that cannot be portably serialized without asking standard librarians to provide extra overhead in their implementations. In this manner, [P1885R8] covers a large number of use cases while leaving niche cases to platform specifics, which is a marked and demonstrable improvement for most individuals who are trying to write Unicode and non-ASCII-friendly software in this day and age.

Asides from this, there are rumblings (particularly in [P2491R0]) that, perhaps, the IANA database set is not enough, or that because IANA specifically states that it is an "octet based" encoding name registry, that each definition there applies to the # of bytes that should logically be represented. This has caused some consternation: for example, if `wchar_t` is represented by a 32-bit number, but someone puts UTF-16 into it, how can it possibly be called "UTF-16" by IANA standards, since it has unused octets in its higher bytes? Does this not make the identification less useful?

I find this characterization to be extremely problematic from the get-go. Not only is using an octet-only definition limited and dangerous for portability (because you now have to expose the rather serious concern that `CHAR_BIT != 8` on all C and C++ platforms), it also forces an extreme amount of duplication throughout the platform. For example, UTF-8 is an 8-bit-byte based encoding. Already, I **know** of platforms that put UTF-8 code units in the lower 8 bits of 32-bit

sized `wchar_ts` on their `CHAR_BIT == 32` chips. Are we going to define "UTF-8.32" to encapsulate this fact? What about every other encoding? Is "ASCII.32" now on the table? What happens if they have `CHAR_BIT == 16`, and put ASCII in their `char`? "UTF7-IMAP.C16"?

Down this direction lies a decimation of usefulness for [\[P1885R8\]](#). Everyone will have to start checking `CHAR_BIT`, and `std::endian`, and SEVERAL other factors to find out if they have the exactly right-sized-and-right-encoded value. It also does not encapsulate existing practice: I already have a counter-example in the Digital Signal Processor I spoke of earlier that puts UTF-8 into 32-bit `wchar_t`. They do not call this "UTF-8.32", it is simply UTF-8 to them. Why?

Because they interpret the encoding form to apply to each code unit, not to each octet.

A code unit-based interpretation is much more fruitful. Strongly in favor.

— Strongly Favor

There is no good solution for providing names for all encodings because the same name has been reused to mean different things. The history of text is full of problems. This proposal however, provides enough information to code that the compiler knows such that literals can actually be decoded.

— Strongly Favor

This paper provides important APIs for identifying standard encodings and is critical for building higher level text processing facilities.

— Strongly Favor

Seems reasonable, but experimenting with this will be required to make sure we got it right.

— Weakly Favor

I have some concerns about coupling ourselves to the IANA database, but it seems that rumours of its death were exaggerated, so maybe it's OK. It's a hard problem, I hope this is the right solution.

— Weakly Favor

The new API gives access to compilation information that is otherwise very difficult to obtain. It can be used efficiently at compile time. It makes effective use of the only recognized registry for naming text encodings in a stable way. The weaknesses in the proposal arise from the fact that the way that encodings are named and specified in the IANA registry do not exactly align with the way that text is modelled in C++, but we've done our best to deal with the incompatibilities. This proposal isn't perfect, but I don't know how to make it any better.

— Weakly Favor

I remain concerned about:

1) use of the IANA character registry as the sole source of named encodings, 2) the special case handling of UTF-16 and UTF-32, 3) the lack of implementation experience in a shipping compiler, 4) how wide character encodings are handled, and 5) whether the proposed `text_encoding` type will be useful for other envisioned encoding identification purposes (e.g., as tag types for encoding sensitive function templates).

I think many of these issues are fixable, so long as they are addressed for C++23.

— Neutral

I don't like the spelling of `text_encoding::id` enumerators; there seems to be no consistency in casing and the style - if a single one even exists - is entirely inconsistent with the rest of the standard library. Sure, it's from the IANA database, but we seem to have no problem doing other messages on the names already since we are dropping the "cs" prefix and renaming two encodings.

— Neutral

This feature is every useful (see for example next poll) and I believe should be included in standard. However, after reading [\[P2491R0\]](#), I agree that this concern need resolution. This may be as simple, as limiting returning IANA schemes to `sizeof(char_size) == 1`, i.e. leaving `wide_literal` implementation defined now.

— Weakly Against

The concerns expressed in [\[P2491R0\]](#) are very real. When `wchar_t` is in use and is of the right size, it is plausible to use names like UTF-16 to refer to an encoding form (trivially encoded as a sequence of `wchar_t` values) rather than an encoding scheme, but that would require weakening the focus on IANA in P1885 and would have only a weak connection to the intended utilities like `iconv` that plainly interpret such names as referring to an encoding scheme. Implementations can specify that functions like `iconv` work with `wchar_t` arrays without (and are not assisted in doing so by) any statement about object representations in the standard.

It is also true that inventiveness in a domain already so chaotic is plainly undesirable. While it might merely reaffirm [\[P1885R8\]](#)'s practical direction with a different interpretation, it seems that we need to reconsider the choices made in the extremely troublesome area of non-octet-based encoding schemes.

— Weakly Against

See [\[P2491R0\]](#) (Text Encodings Follow-Up) for details. In short:

- [\[P1885R8\]](#) re-uses the encoding "UTF16" for a different purpose, which causes confusion.
- [\[P1885R8\]](#) cannot properly represent the UCS-2 wide encoding used on Microsoft Windows before the switch to UTF-16.
- [\[P1885R8\]](#) attempts to specify object representation, which breaks an important C++ abstraction barrier

— Strongly Against

The observations in [\[P2491R0\]](#) are on point. The present proposal disregards an existing ISO standard in favour of "existing practice", which is not a step we should take lightly, and it could easily increase the existing confusing landscape around the terms related to "UTF-16". Finally, [\[P2491R0\]](#)'s point that the library is the wrong place to talk about object representation is serious and warrants more work.

— Strongly Against

§ 3.8. Poll 8: [P2419R1] Clarify Handling Of Encodings In Localized Formatting Of Chrono Types

This proposal relaxes requirements on implementations, giving them the option to provide better results if they want to. It is uniformly applicable across all Unicode encoding forms. It resolves the LWG issue adequately.

— Strongly Favor

This seems to be the least worst solution to the problem, and we need one now.

— Strongly Favor

Looks ok, although I'd prefer solving the general problem of mixing literal and locale-dependent encodings.

— Weakly Favor

Handling a mismatch between the literal encoding and the locale's encoding requires a great deal of machinery and discipline, but since in practice the literal encoding must be presumed for all strings of whichever character type, it is obviously the correct approach to select the characters from the locale and the encoding from the implementation.

— Weakly Favor

I remain concerned about use of the literal encoding as a proxy for the encoding of the output.

— Neutral

§ 3.9. Poll 9: [P2460R0] Relax Requirements On `wchar_t` To Match Existing Practices

Admitting that `wchar_t` is almost always UTF-16, since it's almost never used on POSIX systems, seems long overdue. This is one small thing, but fixing this helps in cleaning up treatment of text literals throughout the standard.

— Strongly Favor

The requirements on `wchar_t` have never been useful in practice and there are platforms where no conforming implementation has ever existed.

— Strongly Favor

Useful relaxation of requirements. We want C++ to be suitable for development on a major platform.

— Weakly Favor

Bringing `wchar_t`'s standardization in line with its actual real world usage is probably a good thing.

— Weakly Favor

There is of course fundamentally no good answer here now. The adoption of 16-bit `wchar_t` on Windows was an attempt to satisfy [basic.fundamental]/8 and was merely overtaken by events. Changing the wording in this trivial fashion does not really fix anything, since there is no reasonable "execution wide-character set" for Windows that satisfies the same outdated requirement. To do better would require a C ABI break that will never happen, or else the provision of a `wchar_t` type and two new sets of C library functions (one for `wchar_t` as UCS-4 and one for `wchar_t` as UTF-16); the only alternative (which is correct but not better) is to simply indicate in the standard that `wchar_t` is just another code-unit type whose library functions are not portably usable.

— Neutral

The standard should take a clear stance on whether UTF-16 as a `wchar_t` encoding is ok or not. If it's ok, a substantial overhaul of the relevant library facilities (some of them inherited from C) is needed, which this paper does not offer. If it's not ok, we don't need this paper. Allowing UTF-16 as a literal encoding (which this paper achieves) but then disallowing that same encoding for the entirety of the standard library is unwise.

— Weakly Against

§ 3.10. Poll 10: [\[P2445R0\]](#) `forward_like`

Already using it, can't imagine this not being in `std::`.

— Strongly Favor

Despite the fact, that this paper introduce 3rd independent way of forwarding in standard, I think that it is necessary. Especially when the number of cases when this is needed will be dramatically increased by deducing this.

— Strongly Favor

The deducing-this feature is, not without reason, quite complicated; we need features like this to minimize the complexity for programmers. The analysis is suitably exhaustive for something that will be used ubiquitously in exactly the situations where all corner cases will be exercised without direct human checking.

— Weakly Favor

I'm really torn on this one because I find its value to be quite a bit overstated, despite having clear use-cases.

— Weakly Favor

The gap this paper fills was not large until the adoption of explicit this parameter.

— Neutral

§ 3.11. Poll 11: [\[P2417R0\]](#) A More `constexpr` `bitset`

Bitsets are important building blocks for low-level applications. The more here is `constexpr`, the better. Especially if you want to gain more foothold on e.g. embedded systems ...

— Strongly Favor

To me bitset is an essential building block for flags/masks that should replace int/enum-based flags. Marking almost all operations of bitset constexpr brings us closer to abandoning "unsafe" practices.

— Strongly Favor

We do constexpr bitset operations, but implemented our own. This may eliminate the need for that.

— Weakly Favor

Everything that can be constexpr should be constexpr.

— Weakly Favor

There's no reason that this shouldn't be usable at compile time.

— Weakly Favor

I think this is a (minor) implementer burden, without providing much value to users. In my experience, bitset isn't used much in the wild.

— Neutral

I was for, but some library implementers pointed out issues with the hash functions, and they are still in the proposal. I'm reluctant to back a library feature our implementers are not comfortable with

— Neutral

§ 3.12. Poll 12: [P1841R1] Wording For Individually Specializable Numeric Traits

The Standard Library desperately needs individual numeric traits that are SFINAE friendly. The new floating-point constants and naming improvements will make it easier to write more portable numerical algorithms for non-IEEE 754 types. I also approve of the removal of `value_exists` and `value_or` from the proposal.

— Strongly Favor

This is quite an improvement on the aging design of `numeric_limits` - which IMHO should be deprecated immediately after this has been adopted.

— Strongly Favor

It is unfortunate to introduce further duplicate facilities into the standard library, but the motivation for doing so in this case is sound. There are several wording issues in the current version, but they can plainly be addressed by LWG with perhaps some minor questions to lib-ext@.

— Weakly Favor

This is yet another work around for having a lack of proper static customization facility in the language.

— Weakly Favor

§ 3.13. Poll 13: [\[P0627R6\]](#) Function To Mark Unreachable Code

Having an explicit, standard means of expressing undefined behavior allows implementations to issue warnings for unconditional undefined behavior while (optionally) emitting traps for statements incorrectly marked unreachable. Anyone who prefers the defined trap in all cases can call `abort`.

— Strongly Favor

I have comprehensive experience with the "compiler magic less" implementation of `unreachable` - this function is essential in documenting impossible values/finding bugs.

— Strongly Favor

This function reflects well what implementations already provide, without overstepping by asking for additional features (see e.g., Section 4.2, "Diagnostic string"). It's a much more readable and teachable interface than `__assume(false)` or the contract form `[[assert axiom: false]]`. Regarding contract forms in general, I like that calling `unreachable` is undefined behavior, whereas failing a contract assertion might not be.

— Strongly Favor

The message based unreachable doesn't seem super important, so I'm fine pushing through unreachable without the message.

— Weakly Favor

§ References

§ Informative References

[P0592R4]

Ville Voutilainen. *To boldly suggest an overall plan for C++23*. 25 November 2019. URL: <https://wg21.link/p0592r4>

[P0627R6]

Jens Maurer. *Function to mark unreachable code*. 25 October 2021. URL: <https://wg21.link/p0627r6>

[P1841R1]

Walter E Brown. *Wording for Individually Specializable Numeric Traits*. 15 May 2020. URL: <https://wg21.link/p1841r1>

[P1885R8]

Corentin Jabot, Peter Brett. *Naming Text Encodings to Demystify Them*. 13 October 2021. URL: <https://wg21.link/p1885r8>

[P2255R2]

Tim Song. *A type trait to detect reference binding to temporary*. 14 October 2021. URL: <https://wg21.link/p2255r2>

[P2387R2]

Barry Revzin. *Pipe support for user-defined range adaptors*. 18 October 2021. URL: <https://wg21.link/p2387r2>

[P2417R0]

Daniil Goncharov. *A more constexpr bitset*. 24 July 2021. URL: <https://wg21.link/p2417r0>

[P2419R1]

Victor Zverovich, Peter Brett. *Clarify handling of encodings in localized formatting of chronotypes*. 19 September 2021. URL: <https://wg21.link/p2419r1>

[P2440R0]

Tim Song. *ranges::iota, ranges::shift_left, and ranges::shift_right*. 13 September 2021. URL: <https://wg21.link/p2440r0>

[P2442R0]

Tim Song. *Windowing range adaptors: views::chunk and views::slide*. 14 September 2021. URL: <https://wg21.link/p2442r0>

[P2443R0]

Tim Song. *views::chunk_by*. 15 September 2021. URL: <https://wg21.link/p2443r0>

[P2445R0]

Gašper Ažman. *forward_like*. 12 October 2021. URL: <https://wg21.link/p2445r0>

[P2454R0]

Bryce Adelstein LeBach. *2021 November Library Evolution Polls*. 3 November 2021. URL: <https://wg21.link/p2454r0>

[P2460R0]

Corentin Jabot. *Relax requirements on wchar_t to match existing practices*. 9 October 2021. URL: <https://wg21.link/p2460r0>

[P2465R1]

Stephan T. Lavavej, Gabriel Dos Reis, Bjarne Stroustrup, Jonathan Wakely. *Standard Library Modules std and std.compat*. 13 October 2021. URL: <https://wg21.link/p2465r1>

[P2485R0]

Jonathan Wakely. *Do not add value_exists and value_or to C++23*. 1 November 2021. URL: <https://wg21.link/p2485r0>

[P2491R0]

Jens Maurer. *Text encodings follow-up*. 15 November 2021. URL: <https://wg21.link/p2491r0>